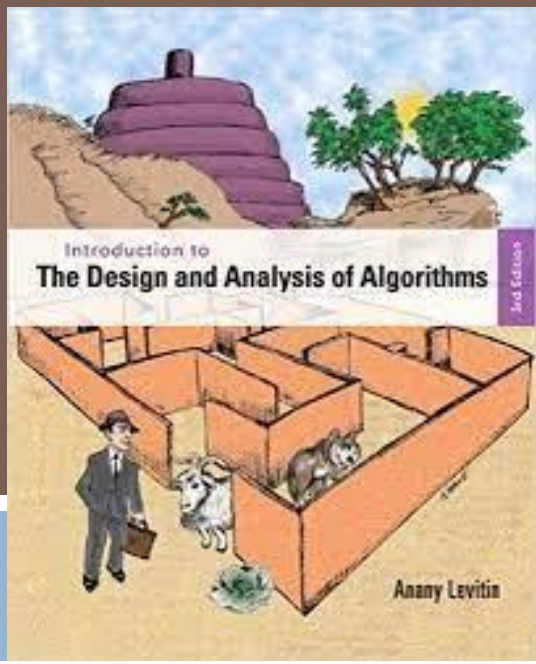
8-Dynamic Programming

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 1 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved



Dynamic Programming

Dynamic Programming is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems

- Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems and later assimilated by CS
- "Programming" here means "planning"

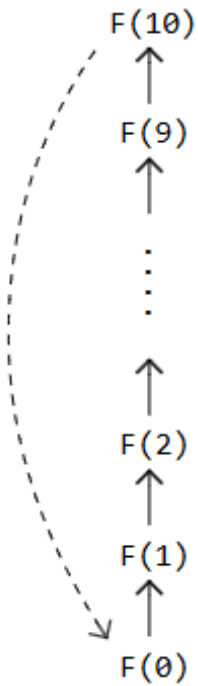
Dynamic Programming

- Main idea:
  - set up a recurrence relating a solution to a larger instance to solutions of some smaller instances
  - solve smaller instances once
  - record solutions in a table
  - extract solution to the initial instance from that table

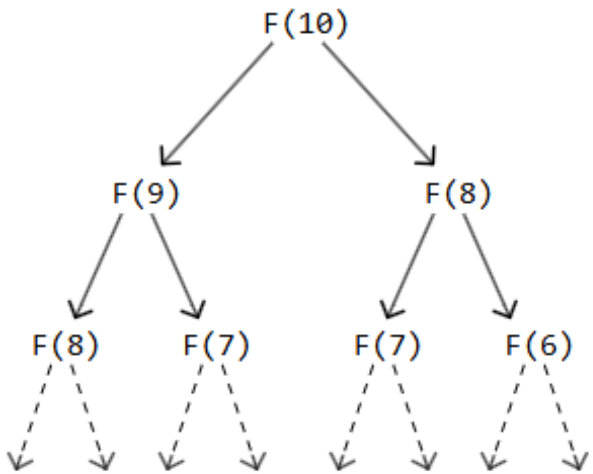
Approaches of Dynamic Programming (DP)

- Dynamic programming can be achieved using two approaches:
  - Top-Down Approach (Memoization)
  - Bottom-Up Approach (Tabulation)

Tabulation and Memoization



The bottom up tabulation approach to finding the 10th Fibonacci number.



The top down recursive approach to finding the 10th Fibonacci number.

Tabulation

- Tabulation uses a table where the results to the most basic subproblems are stored first. The table then gets filled with more and more subproblem results until we find the result to the complete problem that we are looking for
- Generic Chracteristics
  - Bottom-up iteration.
  - No recursion (so no call stack overhead).
  - Solves all subproblems, even if not all are needed.
  - Usually more space-efficient (and can be optimized to O(1) space).

Fibonacci Tabulation


```
public class Main {
    public static int fibonacciTabulation(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        int[] F = new int[n + 1];
        F[0] = 0;
        F[1] = 1;
        for (int i = 2; i <= n; i++) {
            F[i] = F[i - 1] + F[i - 2];
        }
        for (int num : F) {
            System.out.print(num + " ");
        } return F[n];
    }
    public static void main(String[] args) {
        int n = 10;
        int result = fibonacciTabulation(n);
        System.out.println("\nThe " + n + "th Fibonacci number is " + result);
    }
}
```

https://www.w3schools.com/dsa/trydsa.php?filename=demo_findfibo_tab

Memoization

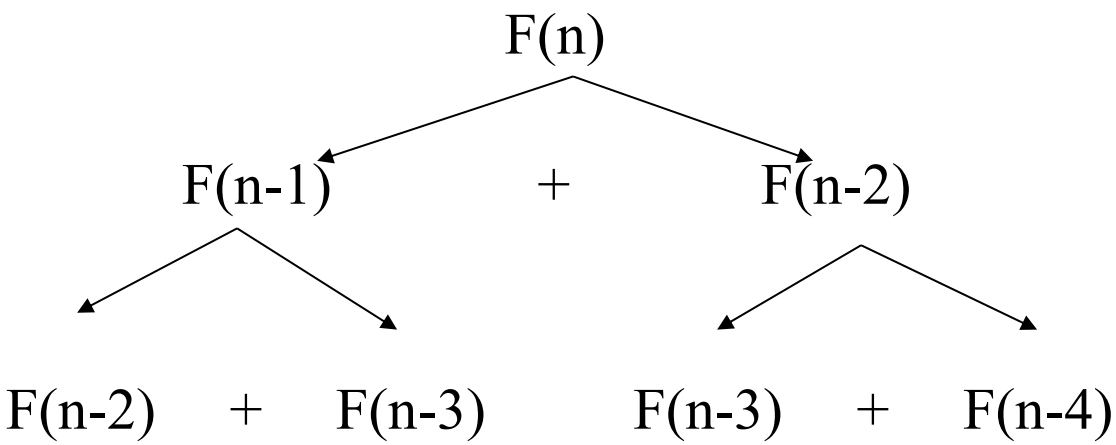
- Keeping the solution recursive and add a memoization table to avoid repeated calls of same subproblems.
  - Before making any recursive call, we first check if the memoization table already has solution for it.
  - After the recursive call is over, we store the solution in the memoization table.

Example 1: Fibonacci numbers

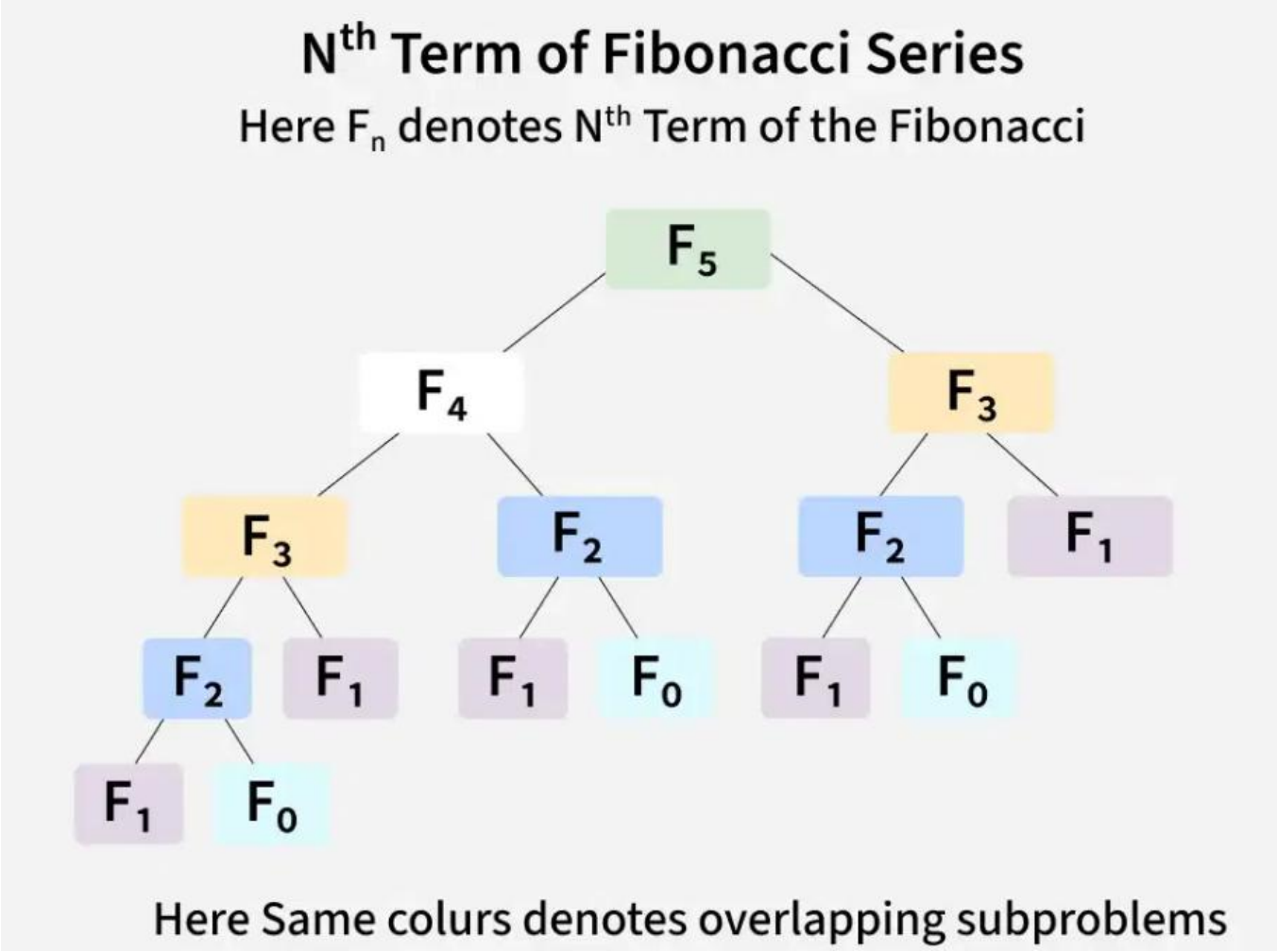
- Recall definition of Fibonacci numbers:

$$F(n) = F(n-1) + F(n-2)$$
$$F(0) = 0$$
$$F(1) = 1$$

- Computing the nth Fibonacci number recursively (top-down):



Fibonnacci


N^th Term of Fibonacci Series
Here F_n denotes N^th Term of the Fibonacci
F5
F4
F3
F3
F2
F2
F1
F2
F1
F1
F0
F1
F0
F1
F0
Here Same colurs denotes overlapping subproblems

Generic Structure


```
if ( n in memo) return memo[n];
   if ( n <=2 ) return 1
   memo[n] = fib(n-1, memo) + fib(n-2, memo)
   return memo[n]
}
```

Example 1: Fibonacci numbers

- Computing the nth Fibonacci number using bottom-up iteration and recording results:

$F(0) = 0$

$F(1) = 1$

$F(2) = 1+0 = 1$

…

$F(n-2) =$

$F(n-1) =$

$F(n) = F(n-1) + F(n-2)$

Efficiency:

- time
- space

The implementation of Fibonacci with memorization

- An memo array initialized to -1. As we make a recursive call, we first check if the value stored in the memo array corresponding to that position is -1.
- The value -1 indicates that we haven't calculated it yet and have to recursively compute it.
- The output must be stored in the memo array so that, next time, if the same value is encountered, it can be directly used from the memo array.

The implementation of Fibonacci with memorization


```
class GfG {
    static int fibRec(int n, int[] memo) {
         if (n <= 1) { return n; }
        // To check if output already exists
        if (memo[n] != -1) { return memo[n];  }
        // Calculate and save output for future use
        memo[n] = fibRec(n - 1, memo) + fibRec(n - 2, memo);
        return memo[n];
    }
    static int fib(int n) {
        int[] memo = new int[n + 1];
        Arrays.fill(memo, -1);
        return fibRec(n, memo);
    }
    public static void main(String[] args) {
        int n = 5;
        System.out.println(fib(n));
    }
}
```

DP Complexity

- Using Tabulation Approach - O(n) Time and O(n) Space
- Assume T(n) is the time complexity of n, so T(n) = T(n-1) + T(n-2).
- Because F(n-2) is in the cache when we calculate F(n - 1), so the operation of F(n-2) is 1(reading from cache), so
- T(n) = T(n-1) + 1 = T(n-2) + 2 = ... = T(n-n) + n. And T(0) is 1, so T(n) = O(n + 1) = O(n).

Finding DP algorithm

- Visualize
- Find an appropriate subprogram
- Find relationship between subprograms
- Generalize relationship
- Implement the solution of subprograms in order.

Example 2: Maximum Non-Adjacent Coin Picking problem

- There is a row of n coins whose values are some positive integers $c_1, c_2, \ldots, c_n$, not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.

- E.g.: 5, 1, 2, 10, 6, 2. What is the best selection?
- If I pick 5, I cannot select 1 5+10+2 =17

- Brute force solution: try all subsets and check the constraint

Classical Recursive Coin Picker


```
public static int solve(int[] coins, int i) {
        if (i >= coins.length) {
            return 0;
        }
        // Option 1: Skip current coin
        int skip = solve(coins, i + 1);

        // Option 2: Take current coin
        int take = coins[i] + solve(coins, i + 2);

        // Return best of two choices
        return Math.max(skip, take);
    }
```

Complexities

- $T(n)=T(n-1)+T(n-2)$ $T(0)=0$,
- And
- Time $O(n) = 2^n$
- Space = n

DP solution to the Maximum Non-Adjacent Coin Picking problem

Let $F(n)$ be the maximum amount that can be picked up from the row of n coins. To derive a recurrence for $F(n)$, we partition all the allowed coin selections into two groups:

those without last coin – the max amount is ?

those with the last coin -- the max amount is ?

DP solution to the Maximum Non-Adjacent Coin Picking problem

Thus, we have the following recurrence

$$F(n) = \max\{c_n + F(n-2), \quad F(n-1)\} \; for \; n > 1,$$

$$F(0) = 0, \quad F(1) = c_1$$

Maximum Non-Adjacent Coin Picking problem

- For coin i, you have two choices:
  - **Do NOT pick coin i**
  - total = dp[i−1]
- **Pick coin i**
  - total = c[i] + dp[i−2]

Example

- Index: 1 2 3 4 5 6
- Coin: 5 1 2 10 6 2

i (coin index)	**coin[i]**		**dp[i-2]**	**dp[i-1]**	Option 1 (skip i)	Option 2 (take i) = coin[i] + dp[i-2]	dp[i]
1	**5**	+	**0**	–	–	5 + 0 = **5**	**5**
2	**1**	+	**0**	**5**	5	1 + 0 = 1	**5**
3	**2**	+	**5**	**5**	5	2 + 5 = **7**	**7**
4	**10**	+	**5**	**7**	7	10 + 5 = **15**	**15**
5	**6**	+	**7**	**15**	15	6 + 7 = **13**	**15**
6	**2**	+	**15**	**15**	15	2 + 15 = **17**	**17**

DP solution to the Maximum Non-Adjacent Coin Picking problem

$F(n) = \max\{c_n + F(n-2), \quad F(n-1)\} \; for \; n > 1,$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5					

$F[0] = 0, F[1] = c_1 = 5$

F(2) = max{F(0)+C2,F(1)}=max{0+1,5} = 5

F(3) = max{F(1)+C3,F(2)}=max{5+2,5} = 7

F(4) = max{F(2)+C4,F(3)}=max{5+10,7} = 15

F(5) = max{F(3)+C5,F(4)}=max{7+6,15} = 15

F(6) = max{F(4)+C6,F(5)}=max{15+2,15} = 17

DP solution to the Maximum Non-Adjacent Coin Picking problem

$F[0] = 0,\ F[1] = c_1 = 5$

index	0	1	2	3	4	5	6
C		5	1	2	10	6	2
F	0	5	5	7	15	15	17

Max amount: 17

Coins of optimal solution:

backtracking.

$f(4) + c_6$

$f(2) + c_4$

$f(1)$

c_1

$\Rightarrow c_1, c_4, c_6$

Time efficiency: $\Theta(n)$

Space efficiency: $\Theta(n)$

Note: All smaller instances were solved

Implementation


```
public static int maxCoins(int[] coins) {
    int n = coins.length;
    if (n == 0) return 0;
    if (n == 1) return coins[0];

    int[] dp = new int[n];
    dp[0] = coins[0];
    dp[1] = Math.max(coins[0], coins[1]);

    for (int i = 2; i < n; i++) {
        dp[i] = Math.max(dp[i - 1], coins[i] + dp[i - 2]);
    }

    return dp[n - 1];
}
```

Memoization version


```
int maxCoinSum(int i) {
    if (i < 0) return 0;        // no coins left
    if (memo[i] != -1) return memo[i]; // already computed

    // Pick coin i or skip it
    int pick = coins[i] + maxCoinSum(i - 2);
    int skip = maxCoinSum(i - 1);

    memo[i] = Math.max(pick, skip);
    return memo[i];
}
```

Compare step by step with DP version


```
public static int solve(int[] coins, int i) {
    if (i >= coins.length) {
        return 0;
    }
    // Option 1: Skip current coin
    int skip = solve(coins, i + 1);

    // Option 2: Take current coin
    int take = coins[i] + solve(coins, i + 2);

    // Return best of two choices
    return Math.max(skip, take);
}
```

```
int maxCoinSum(int i) {
    if (i < 0) return 0;        // no coins left
    if (memo[i] != -1) return memo[i]; //computed

    // Pick coin i or skip it
    int pick = coins[i] + maxCoinSum(i - 2);
    int skip = maxCoinSum(i - 1);

    memo[i] = Math.max(pick, skip);
    return memo[i];
}
```

Example 3: The Coin Change(Combination) Problem With Dynamic Programming

- Given a list of coins i.e 1 cents, 5 cents and 10 cents, can you determine the total number of combinations of the coins in the given list to make up the number
- Input: N=8
- Coins : 1, 5, 10
- Output: 2

- Explanation:
- 1 way:
- 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 8 cents.
- 2 way:
- 1 + 1 + 1 + 5 = 8 cents.

https://www.geeksforgeeks.org/dsa/understanding-the-coin-change-problem-with-dynamic-programming/

Coin row problem initialize

- We can determine the number of ways the coins make up the values at the index of **Array of ways**.
- we can determine a coin is larger than that value at the index then clearly we can't use that coin to determine the combinations of the coins because that coin is larger than that value
- N = 12
- Index of Array of Coins: [0, 1, 2]
- Array of coins: [1, 5, 10]

- Index of Array of ways:

  [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
- Array of ways:

  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Coin row problem steps

- Then compare coins[0] … coin[n] to all of the index's of ways array. If the value of the coin is less than or equal to the ways index, then ways[j-coins[i]]+ways[j] is the new value of ways[j].
- We do this because we are trying to break each part down into smaller pieces.
- We must set the first element at index 0 of the ways array to 1. This is because there is 1 way to make the number 0, using 0 coins.
- **Index of Array of ways:**
- [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
- **Array of ways:**
- [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Coin row problem steps

- Compare the second coin, 5 cents.
- Comparing 5 cents to each of the index and making that same comparison, if the value of the coin is smaller than the value of the index at the ways array then
- ways[j-coins[i]]+ways[j] is the new value of ways[j]. Thus we get the following.

- Index of Array of ways:
- [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
- Array of ways:
- [1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3]

Coin row problem steps

- Comparing 10 cents

- Index of Array of ways:
- [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
- Array of ways:
- [1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 4, 4, 4]

- So the answer to the example is ways[12] which is 4.

The algorithm in java


```
static long getNumberOfWays(long N, long[] Coins)
  {
     long[] ways = new long[(int)N + 1]; // Create the ways array to 1 plus the amount to stop overflow
    // Set the first way to 1 because its 0, and there is 1 way to make 0 with 0 coins
    ways[0] = 1;
     // Go through all of the coins
    for (int i = 0; i < Coins.length; i++) {
      // Make a comparison to each index value of ways with the coin value.
      for (int j = 0; j < ways.length; j++) {
        if (Coins[i] <= j) {
       // Update the ways array
          ways[j] += ways[(int)(j - Coins[i])];
        }
      }
    }
    return ways[(int)N];
  }
```

The time complexity of this implementation

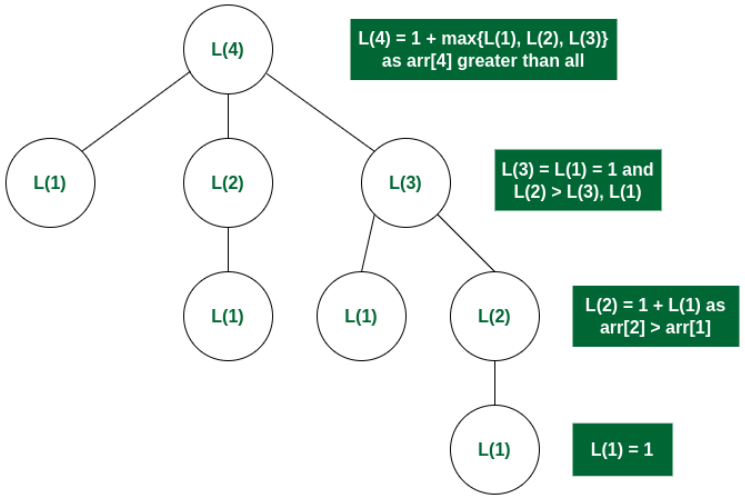
- The time complexity of this implementation is
- O(N * C), where N is the target value and C is the number of coins.
- This is because we have two nested loops: one that iterates through each coin, and another that iterates through each value up to N.

Longest Increasing Subsequence (LIS)

- Given an array arr[] of size n, the task is to find the length of the Longest Increasing Subsequence (LIS) i.e., the longest possible subsequence in which the elements of the subsequence are sorted in increasing order.

LIS

- [Naive Approach] Using Recursion - Exponential Time and Linear Space
- The idea to do traverse the input array from left to right and find length of the Longest Increasing Subsequence (LIS) ending with every element arr[i]. Let the length found for arr[i] be L[i]. At the end we return maximum of all L[i] values.



https://www.geeksforgeeks.org/dsa/longest-increasing-subsequence-dp-3/

LIS

For a sequence $a_1, a_2, \ldots, a_n$, find the length of the longest increasing subsequence $a_{i_1}, a_{i_2}, \ldots, a_{i_k}$

Constraints: $i_1 < i_2 < \cdots < i_k$; $a_{i_1} < a_{i_2} < \cdots < a_{i_k}$

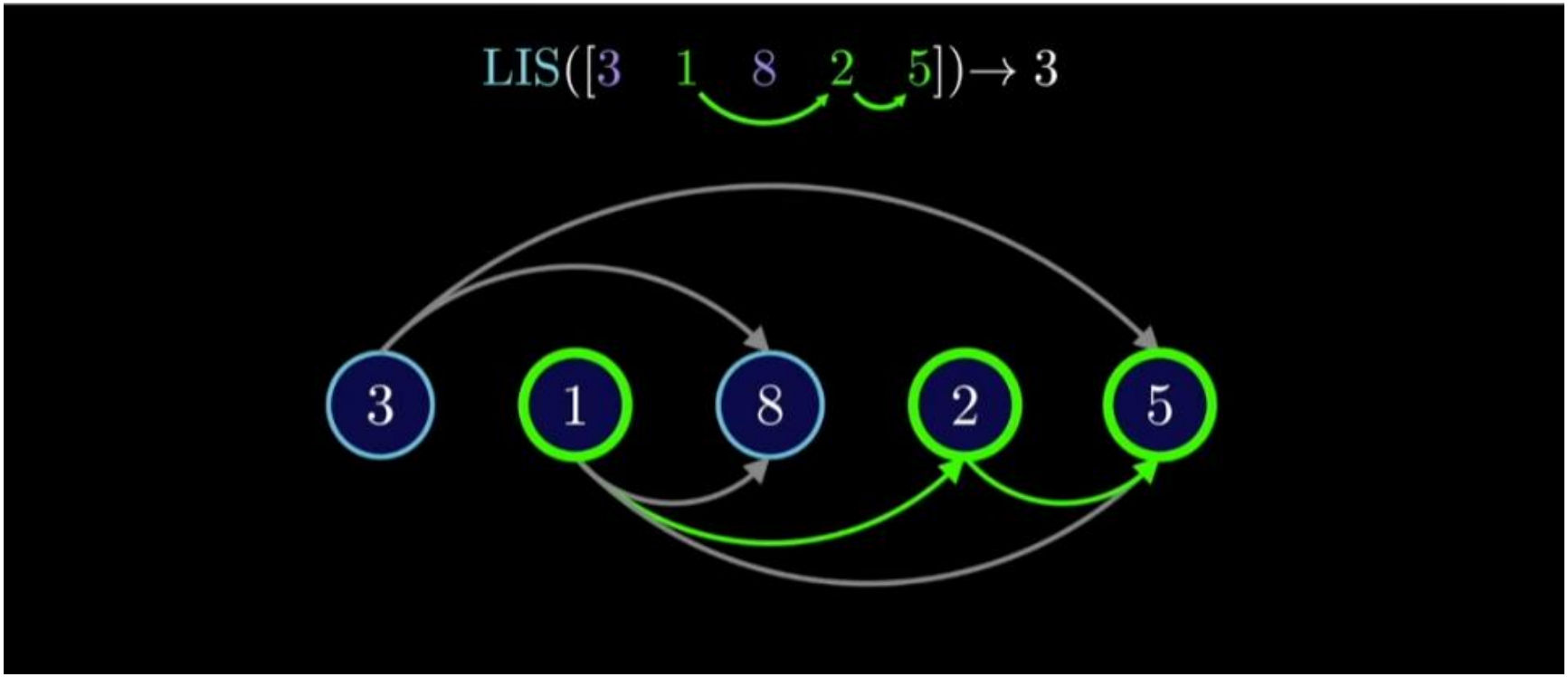
$\text{LIS}([3 \quad 1 \quad 8 \quad 2 \quad 5]) \rightarrow 3$

$\text{LIS}([5 \quad 2 \quad 8 \quad 6 \quad 3 \quad 6 \quad 9 \quad 5]) \rightarrow 4$

We will focus on the length of the LIS

https://www.youtube.com/watch?v=aPQY__2H3tE

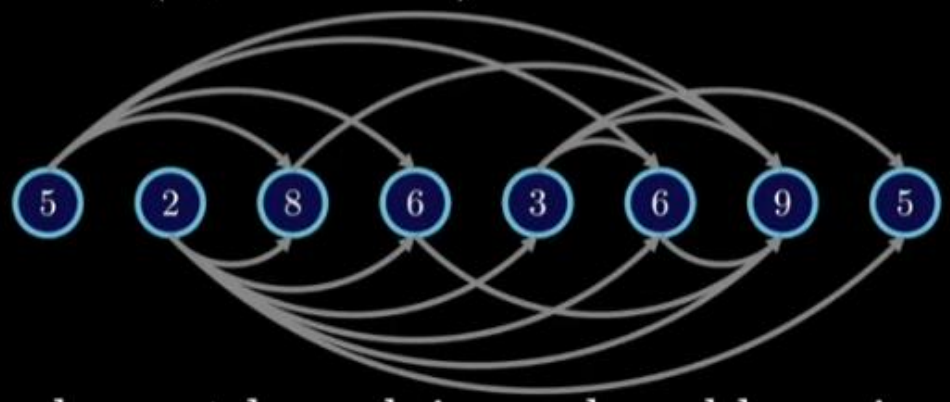
LIS



LIS

$$\mathrm{LIS}[n] = 1 + \max\{\mathrm{LIS}[k] \mid k < n, A[k] < A[n]\}$$

```
def lis(A) :
    L = [1] * len(A)
    for i in range(1, len(L)):
        subproblems = [L[k] for k in range(i) if A[k] < A[i]]
        L[i] = 1 + max(subproblems, default=0)
    return max(L, default=0)
```



5. Implement by solving subproblems in order

LIS Solution


```
public class LIS {
    public static int lis(int[] arr) {
        int n = arr.length;
        int[] dp = new int[n];
        for (int i = 0; i < n; i++) dp[i] = 1; //Each element size is one
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (arr[j] < arr[i]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
        }
        int max = 1;
        for (int x : dp) max = Math.max(max, x);
        return max;
    }
}
```

LIS Solution with Binary Search

- Finding a subsequence by maintaining a list of "buckets" where each bucket represents a valid subsequence. Initially, we start with an empty list and iterate through the input vector arr from left to right.
- For each number in arr, the following steps are performed:
  - If the number is greater than the last element of the last bucket (i.e., the largest element in the current subsequence), we append the number to the end of the list. This indicates that we have found a longer subsequence.
  - Otherwise, we perform a binary search on the list of buckets to find the smallest element that is greater than or equal to the current number. This step helps us maintain the property of increasing elements in the buckets.
  - Once we find the position to update, we replace that element with the current number. This keeps the buckets sorted and ensures that we have the potential for a longer subsequence in the future.

LIS Solution with Binary Search Illustration

- Example: arr = [3, 4, 5, 1, 2, 3, 4] Let's see why keeping 1 (the smallest value) helps:

- We use binary search to find the position where new element is to be inserted.

- First three elements: buckets = [3, 4, 5]
- arr[3] = 1 buckets = [1, 4, 5] // 1 replaces 3
- arr[4] = 2 buckets = [1, 2, 5] // 2 replaces 4 as it's smaller
- arr[5] = 3 buckets = [1, 2, 3] // 3 replaces 5
- arr[6] = 4 buckets = [1, 2, 3, 4] // 4 is appended as it's larger

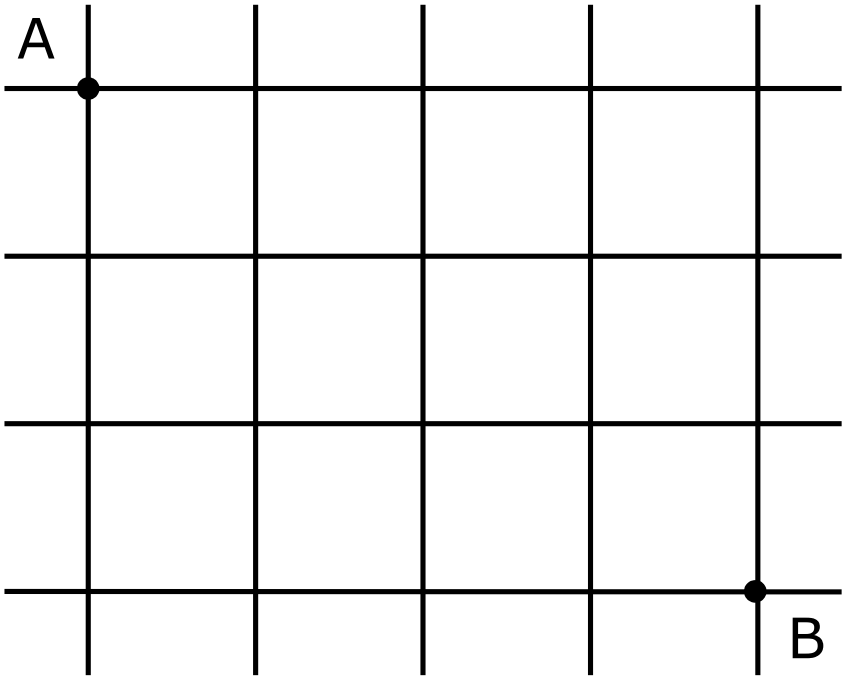
LIS Solution with Binary Search Implementation


```
static int lengthOfLIS(int[] arr) {
    int n = arr.length;
    List<Integer> ans = new ArrayList<>();
    ans.add(arr[0]); // Initialize the answer list with the first element of arr
    for (int i = 1; i < n; i++) {
      if (arr[i] > ans.get(ans.size() - 1)) {
        // If the current number is greater than the last element of the answer list, we have found a longer increasing
            subsequence and append the current number  to the answer list.
        ans.add(arr[i]);
      } else {
        // If the current number is not  greater than the last element of the answer list,
        int low = Collections.binarySearch(ans, arr[i]);
        // Update the element at the found position with the current number to maintain a sorted order in the answer list.
        if (low < 0) {low = -(low + 1); }
        ans.set(low, arr[i]);
      }
    }
    // The size of the answer list represents the length of the longest increasing subsequence.
    return ans.size();
  }
}
```

Example 3: Path counting

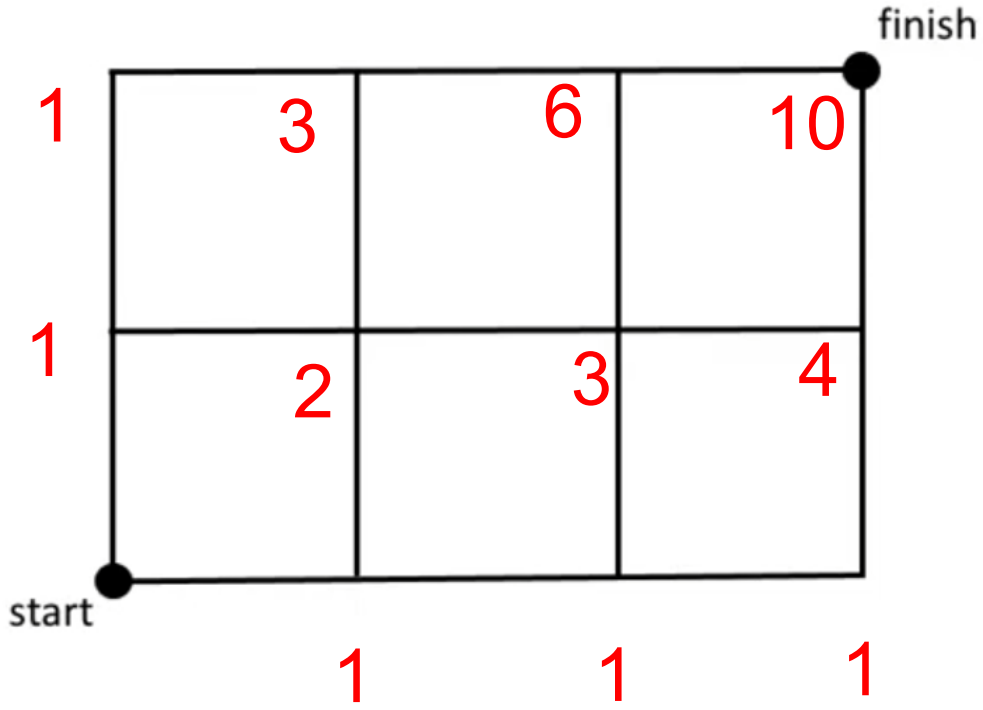
Consider the problem of counting the number of shortest paths from point A to point B in a city with perfectly horizontal streets and vertical avenues

- Only travel down ⇩ and rigth ⇨



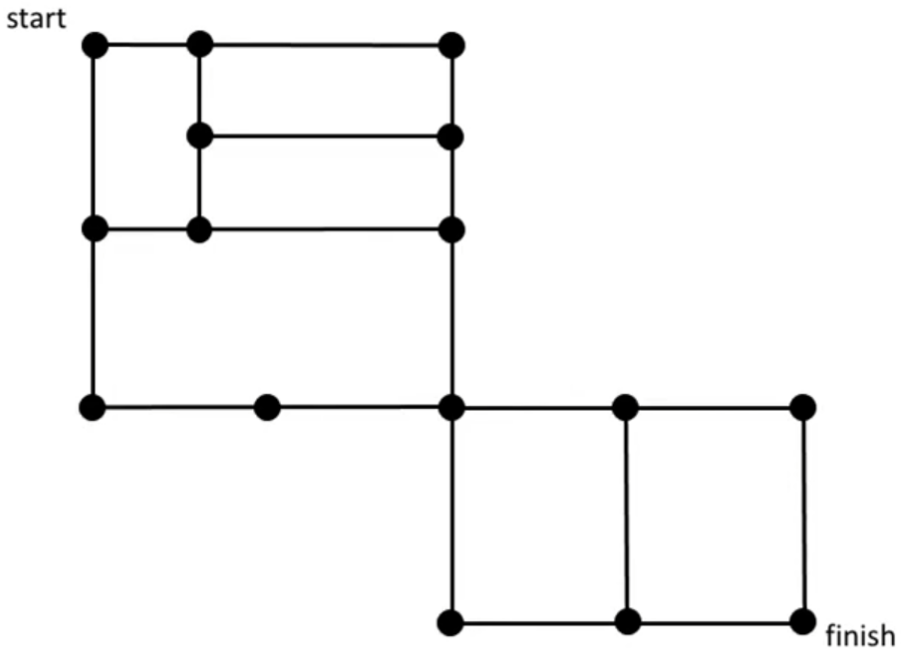
Path Counting

- Initialize the single path opportunities to 1
- Add intersections from low and right

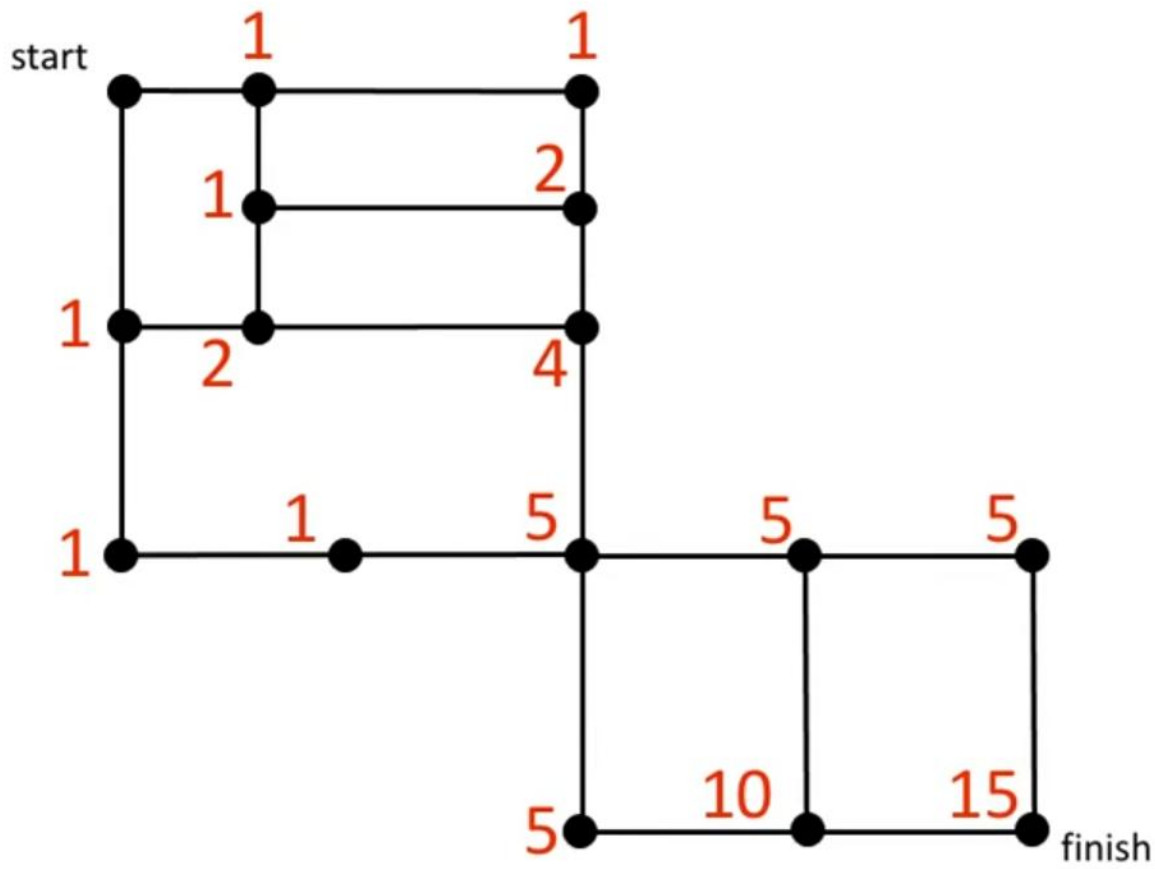


Path Counting

- Travel down and right

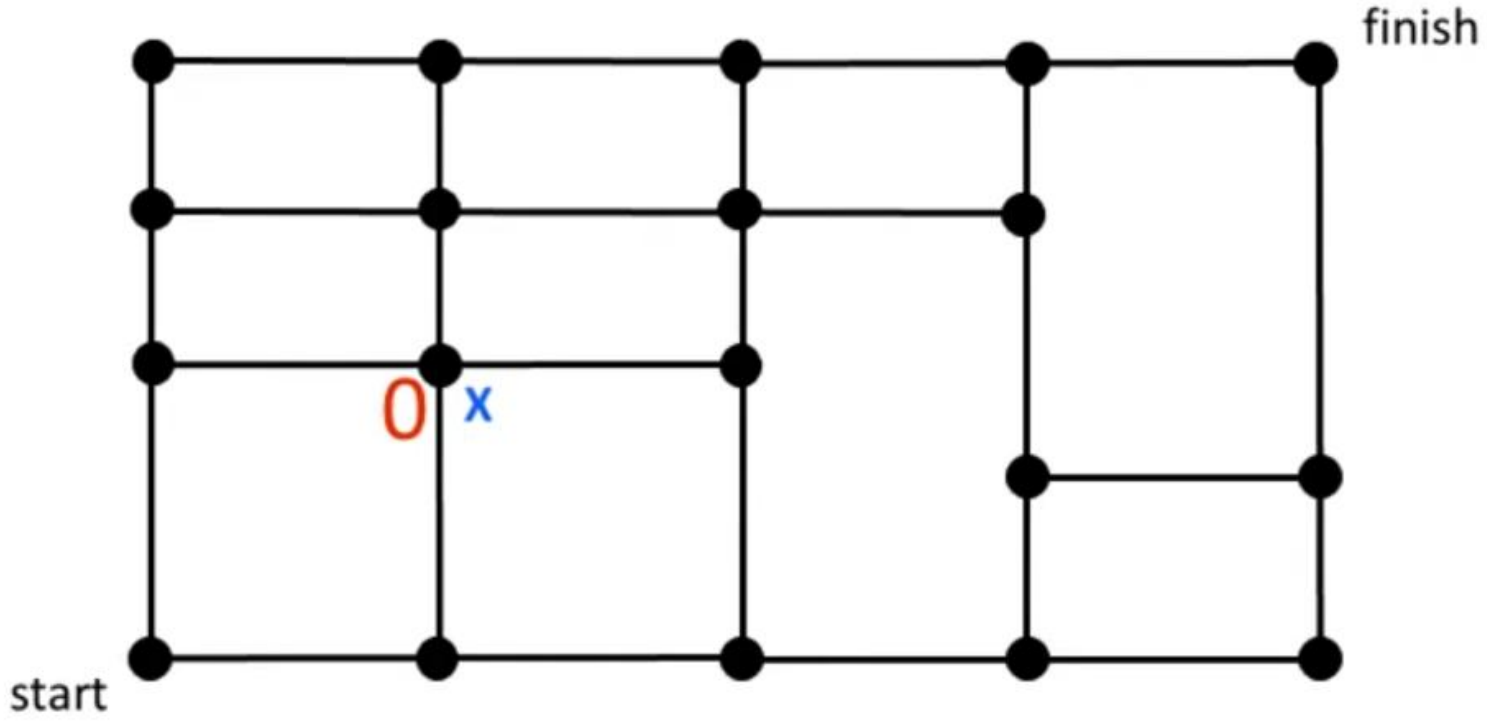


Path Counting

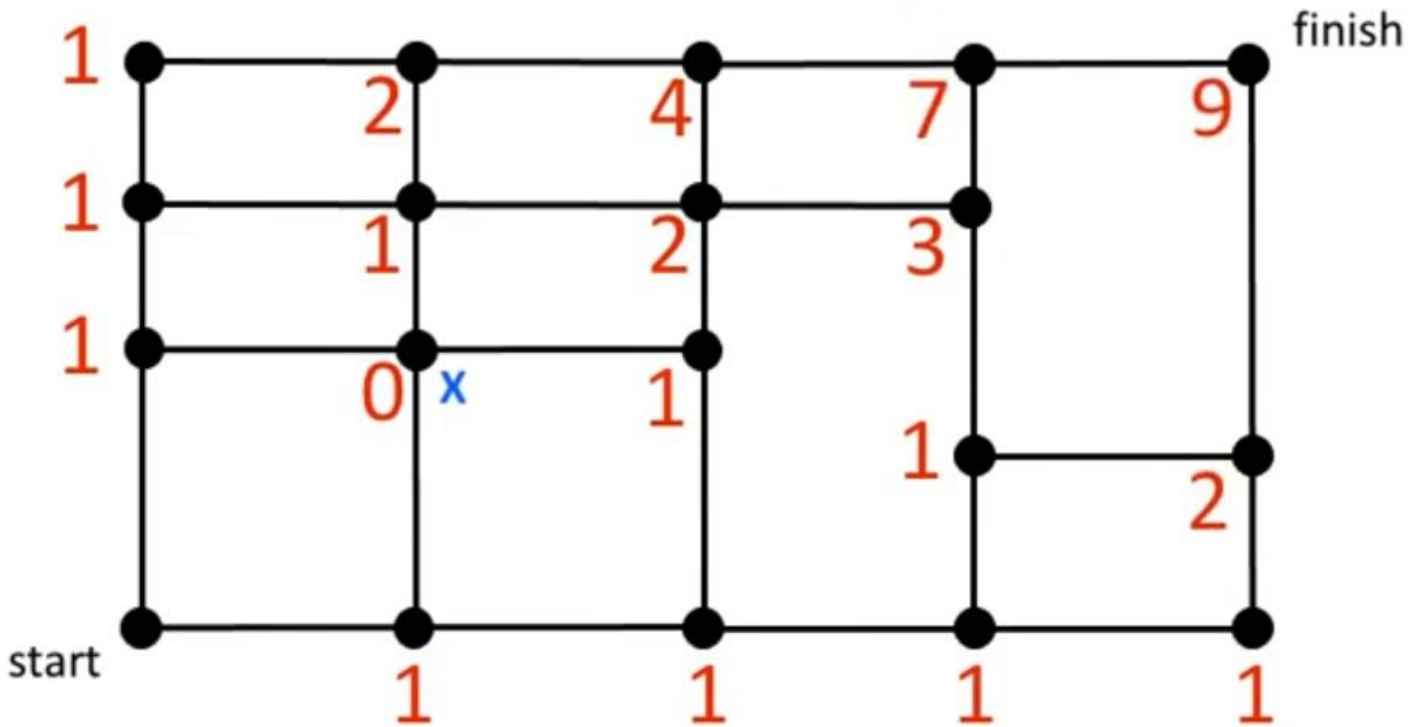


Avoid some path

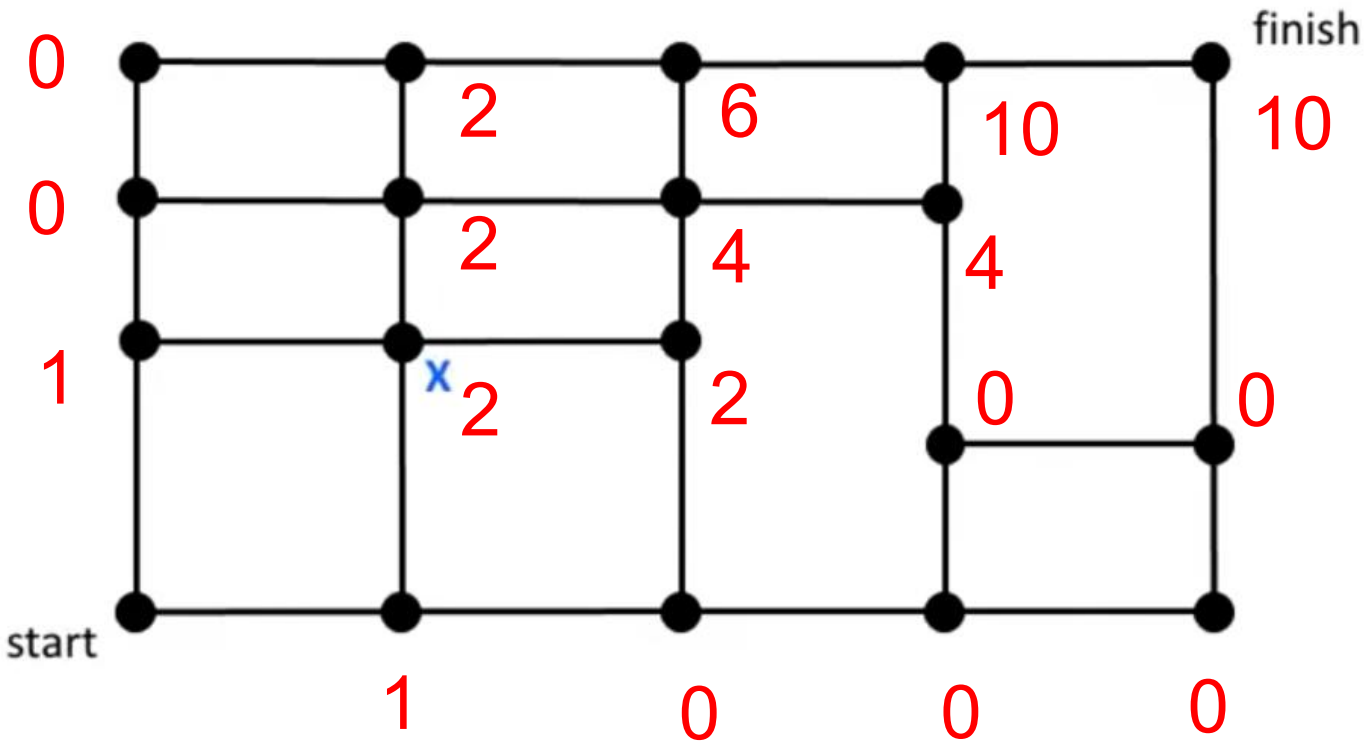
- Replace 0 for the path to avoid



Avoid some paths



Include some path



Path Counting Algorithm


```
public static int countPaths(int m, int n) {
    int[][] dp = new int[m][n];

    // base row & column
    for (int i = 0; i < m; i++) dp[i][0] = 1;
    for (int j = 0; j < n; j++) dp[0][j] = 1;

    for (int i = 1; i < m; i++) {
      for (int j = 1; j < n; j++) {
        dp[i][j] = dp[i-1][j] + dp[i][j-1];
      }
    }

    return dp[m-1][n-1];
  }
```

Complexities

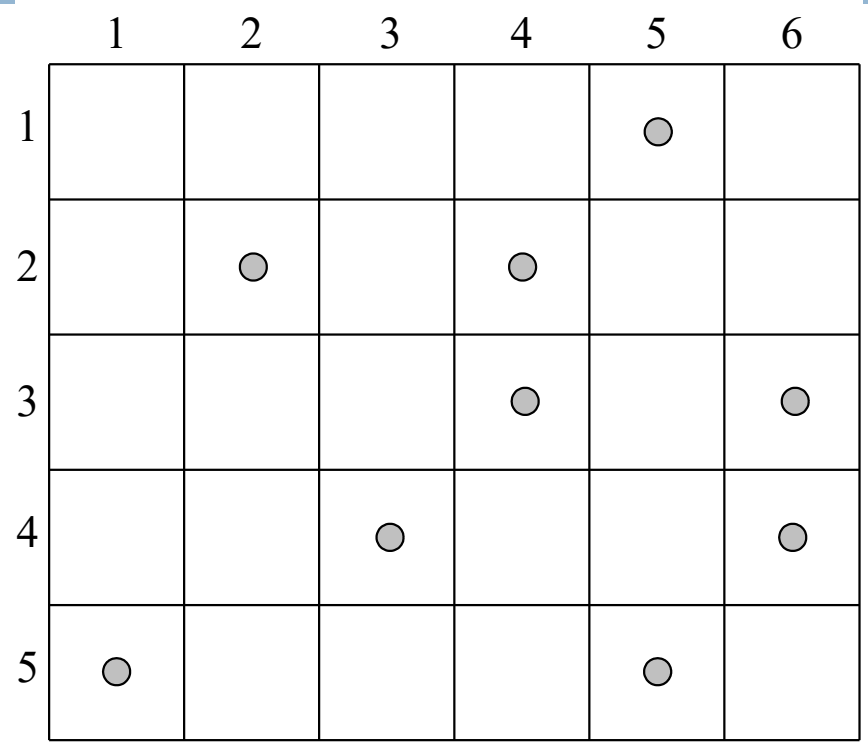
- Grid (Right + Down) $O(mn)$ $O(mn) \rightarrow O(n)$
- Grid with obstacles $O(mn)$ $O(mn) \rightarrow O(n)$

Example 4: Coin-collecting by robot

Several coins are placed in cells of an $n \times m$ board.

A robot, located in the upper left cell of the board, needs to collect as many of the coins as possible and bring them to the bottom right cell.

On each step, the robot can move either one cell to the right or one cell down from its current location.



Solution to the coin-collecting problem

Let $F(i,j)$ be the largest number of coins the robot can collect and bring to cell (i,j) in the ith row and jth column.

The largest number of coins that can be brought to cell (i,j):

from the left neighbor ?

from the neighbor above?

The recurrence:

$$F(i,j) = \max\{F(i-1,j),\ F(i,j-1)\} + c_{ij} \quad for\ 1 \le i \le n, 1 \le j \le m$$

where $c_{ij} = 1$ if there is a coin in cell (i,j), and $c_{ij} = 0$ otherwise

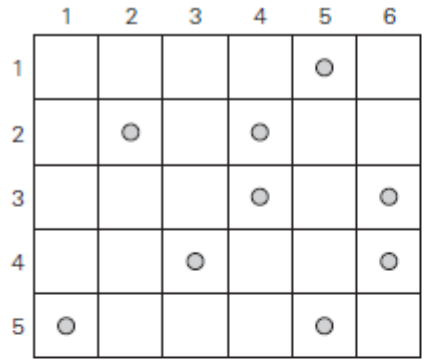
$$F(0,j) = 0\ for\ 1 \le j \le m \text{ and } F(i,0) = 0\ for\ 1 \le i \le n.$$

Solution to the coin-collecting problem

(a) Coins to collect.

(b) Dynamic programming algorithm results.

(c) Two paths to collect 5 coins the maximum number of coins possible.



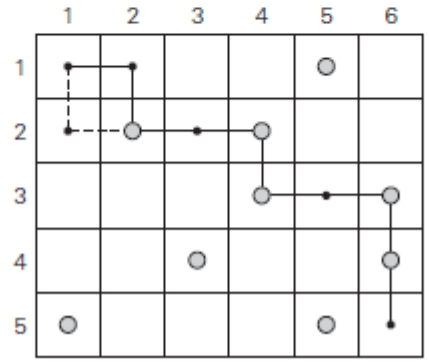
(a)

	1	2	3	4	5	6
1	0	0	0	0	1	1
2	0	1	1	2	2	2
3	0	1	1	3	3	4
4	0	1	2	3	3	5
5	1	1	2	3	4	**5**

(b)

$$F(i, j) = \max\{F(i-1, j), F(i, j-1)\} + c_{ij} \quad \text{for } 1 \le i \le n, \ 1 \le j \le m$$

$$F(0, j) = 0 \ \text{ for } 1 \le j \le m \quad \text{and} \quad F(i, 0) = 0 \ \text{ for } 1 \le i \le n,$$



(c)

Complexities

- Time Complexity:You compute each cell once.
  - $O(m \times n)$
- Space Complexity:With full DP table:
  - $O(m \times n)$

Other examples of DP algorithms

- Computing a binomial coefficient (# 9, Exercises 8.1)
- General case of the change making problem (Sec. 8.1)
- Some difficult discrete optimization problems:
  - knapsack (Sec. 8.2)
  - traveling salesman
- Constructing an optimal binary search tree (Sec. 8.3)
- Warshall's algorithm for transitive closure (Sec. 8.4)
- Floyd's algorithm for all-pairs shortest paths (Sec. 8.4)

Knapsack Problem by DP

Given n items of

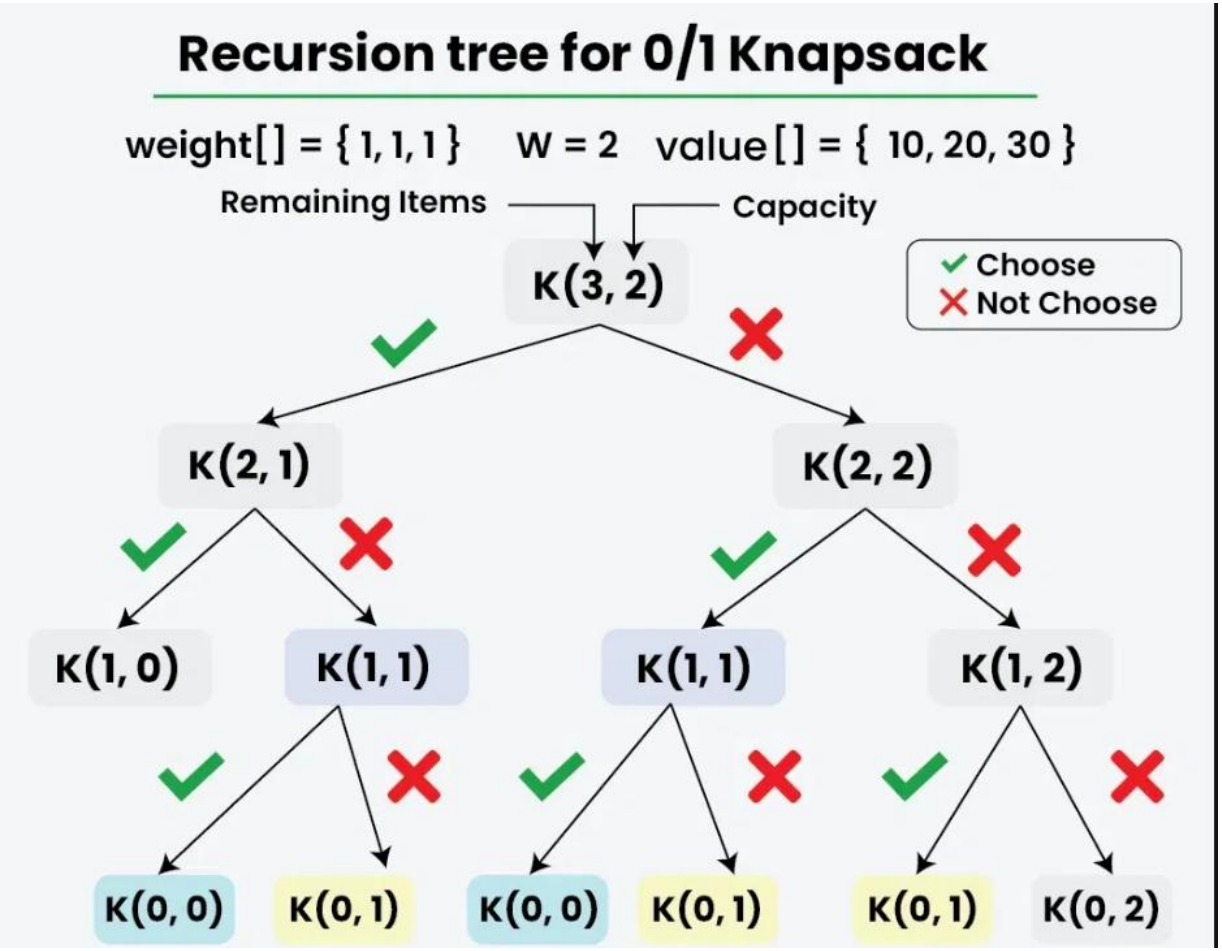
integer weights: $w_1 \quad w_2 \quad \dots \quad w_n$

values: $v_1 \quad v_2 \quad \dots \quad v_n$

a knapsack of integer capacity W

find most valuable subset of the items that fit into the knapsack

Knapsack Problem



The naive function using recursion computes the same subproblems again and again. See the following recursion tree, K(1, 1) is being evaluated twice. https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/

Knapsack Problem by DP

Consider instance defined by first i items and capacity j ($j \leq W$).

Let $F(i, j)$ be the value of an optimal solution to this instance, i.e., the value of the most valuable subset of the first i items that fit into the knapsack of capacity j.

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

Initial conditions:

$$F(0, j) = 0 \text{ for } j \geq 0 \quad \text{and} \quad F(i, 0) = 0 \text{ for } i \geq 0.$$

		0	$j-w_i$	j	W
	0	0	0	0	0
	$i-1$	0	$F(i-1, j-w_i)$	$F(i-1, j)$	
w_i, v_i	i	0		$F(i, j)$	
	n	0			goal

Knapsack Problem by DP

- In the previous solution, we used a n * W matrix. We can reduce the used extra space. The idea behind the optimization is, to compute mat[i][j], we only need solution of previous row.
- In 0-1 Knapsack Problem if we are currently on mat[i][j] and we include ith element then we move j-wt[i] steps back in previous row and if we exclude the current element we move on jth column in the previous row.
- So here we can observe that at a time we are working only with 2 consecutive rows.

Knapsack Problem by DP

If the weight of the item w[i] > j (current capacity), we cannot take it:

- $dp[i][j]=dp[i-1][j]$

Otherwise, we can either take it or not:

- $dp[i][j]=\max(dp[i-1][j],v[i]+dp[i-1][j-w[i]])$

Knapsack Problem by DP (example)

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

$$F(0, j) = 0 \text{ for } j \geq 0 \quad \text{and} \quad F(i, 0) = 0 \text{ for } i \geq 0.$$

item	weight	value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

capacity $W = 5$.

	i	capacity j: 0	1	2	3	4	5
	0	0	0	0	0	0	0
$w_1 = 2,\ v_1 = 12$	1	0	0	12	12	12	12
$w_2 = 1,\ v_2 = 10$	2	0	10	12	22	22	22
$w_3 = 3,\ v_3 = 20$	3	0	10	12	22	30	32
$w_4 = 2,\ v_4 = 15$	4	0	10	15	25	30	37

Knapsack by DP example

- weights = [1, 3, 4, 5]
- values = [1, 4, 5, 7]
- capacity = 7

$$F(i, j) = \begin{cases} \max\{F(i-1, j), v_i + F(i-1, j-w_i)\} & \text{if } j - w_i \geq 0, \\ F(i-1, j) & \text{if } j - w_i < 0. \end{cases}$$

Item\Capacity	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
	0	1						
	0	1						
	0	1						
	0	1						

Knapsack by DP example

- weights = [1, 3, 4, 5]
- values = [1, 4, 5, 7]
- capacity = 7

Item\Capacity	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1 (w=1,v=1)	0	1	1	1	1	1	1	1
2 (w=3,v=4)	0	1	1	4	5	5	5	5
3 (w=4,v=5)	0	1	1	4	5	6	6	9
4 (w=5,v=7)	0	1	1	4	5	7	8	9

Knapsack by DP example
Which objects should be included

- weights = [1, 3, 4, 5]
- values = [1, 4, 5, 7]
- capacity = 7

Item\Capacity	0	1	2	3	4	5	6	7
0	0	0	**0**	0	0	0	0	0
1 (w=1,v=1)	0	1	1	1	1	1	1	1
2 (w=3,v=4)	0	1	1	**4**	5	5	5	5
3 (w=4,v=5)	0	1	1	**4**	5	6	6	**9**
4 (w=5,v=7)	0	1	1	4	5	7	8	9

Another Example

$m=8$ $\quad P=\{1, 2, 5, 6\}$

$n=4$ $\quad \omega=\{2, 3, 4, 5\}$

P_i	ω_i	V	0	1	2	3	4	5	6	7	8
		0	0	0	0	0	0	0	0	0	0
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

x_1	x_2	x_3	x_4
0	1	0	1

$8-6=2$

$2-2=0$

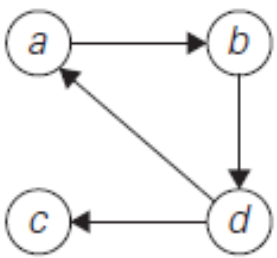
https://www.youtube.com/watch?v=nLmhmB6NzcM

Warshall's and Floyd's Algorithms

- Warshall's algorithm for computing the transitive closure of a directed graph
- Floyd's algorithm for the all-pairs shortest-paths problem.
- These algorithms are based on essentially the same idea:
  - exploit a relationship between a problem and its simpler rather than smaller version

Warshall's Algorithm: Transitive Closure

DEFINITION The ***transitive closure*** of a directed graph with n vertices can be defined as the $n \times n$ boolean matrix $T = \{t_{ij}\}$, in which the element in the ith row and the jth column is 1 if there exists a nontrivial path (i.e., directed path of a positive length) from the ith vertex to the jth vertex; otherwise, t_{ij} is 0.



(a)

$$A = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

(b)

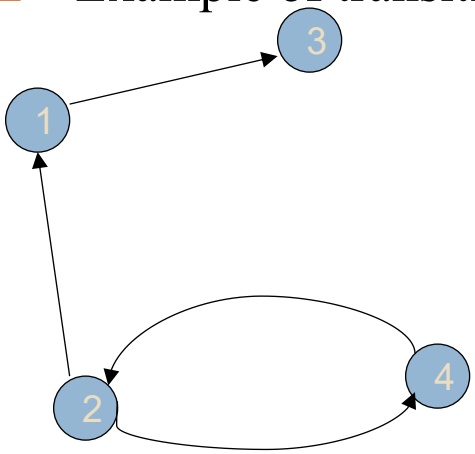
$$T = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{array}$$

(c)

(a) Digraph. (b) Its adjacency matrix. (c) Its transitive closure.

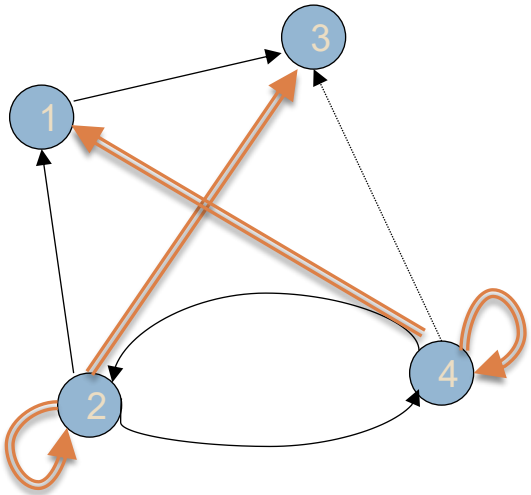
Warshall's Algorithm: Transitive Closure

- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph
- Example of transitive closure:



$$\begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$

adjacency matrix

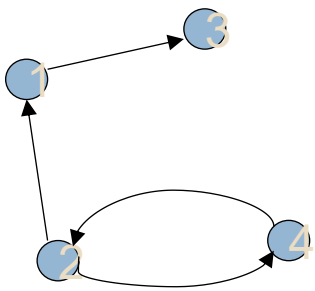


$$\begin{matrix} 0 & 0 & 1 & 0 \\ 1 & \mathbf{1} & \mathbf{1} & 1 \\ 0 & 0 & 0 & 0 \\ \mathbf{1} & 1 & \mathbf{1} & \mathbf{1} \end{matrix}$$

Warshall's Algorithm: Transitive Closure

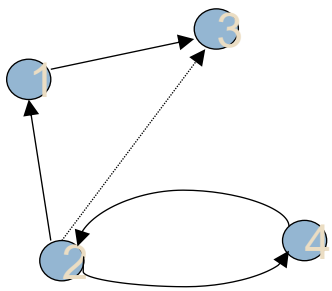
- Constructs transitive closure T as the last matrix in the sequence of n-by-n matrices $R^{(0)}, \ldots, R^{(k)}, \ldots, R^{(n)}$ where $r_{ij}^{(k)} = 1$ iff there is nontrivial path **from i to j with only first k vertices allowed as intermediate**
- Note that $R^{(0)} = \boldsymbol{A}$**(adjacency matrix),** $\boldsymbol{R^{(n)} = T}$ **(transitive closure)**

Warshall's Algorithm: Transitive Closure



$R^{(0)}$

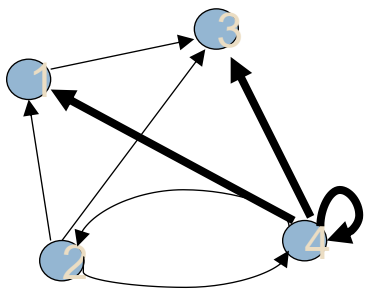
0	0	1	0
1	0	0	1
0	0	0	0
0	1	0	0



$R^{(1)}$

0	0	1	0
1	0	**1**	1
0	0	0	0
0	1	0	0

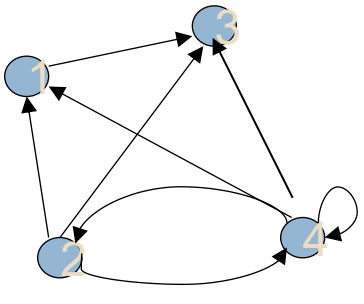
Only node 1 can be used as the intermediate node



$R^{(2)}$

0	0	1	0
1	0	1	1
0	0	0	0
1	1	**1**	**1**

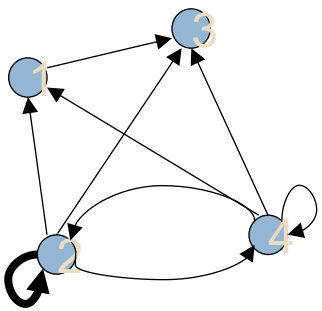
Only node 2



$R^{(3)}$

0	0	1	0
1	0	1	1
0	0	0	0
1	1	1	1

Only node 3



$R^{(4)}$

0	0	1	0
1	**1**	1	1
0	0	0	0
1	1	1	1

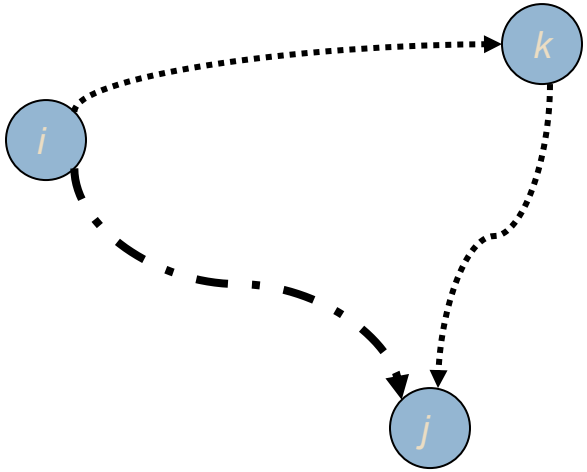
Only node 4

Transitive Closure

Warshall's Algorithm (recurrence)

On the k-th iteration, the algorithm determines for every pair of vertices i, j if a path exists from i and j with just vertices 1,…,k allowed as intermediate.

We have just proved is that if $r_{ij}^{(k)} = 1$, then either $r_{ij}^{(k-1)} = 1$ or both $r_{ik}^{(k-1)} = 1 \; and \; r_{kj}^{(k-1)} = 1$



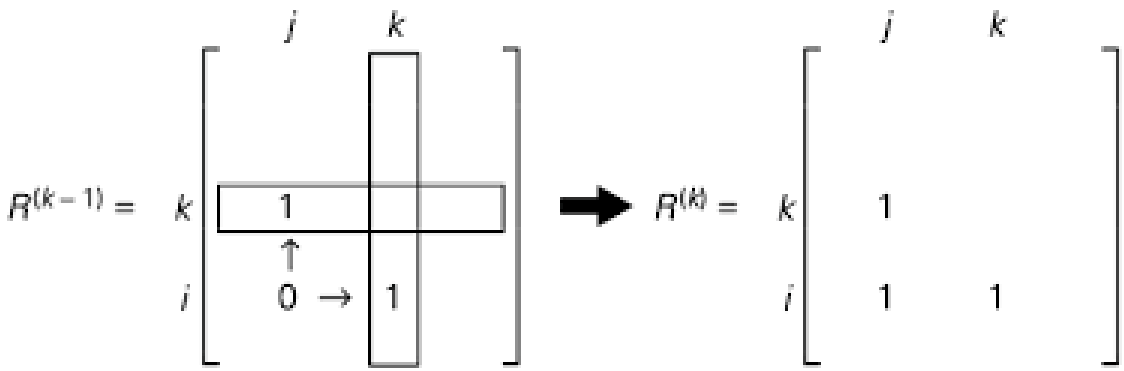
Recurrence relating elements $R^{(k)}$ to elements of $R^{(k-1)}$ is:

$$r_{ij}^{(k)} = r_{ij}^{(k-1)} \quad \text{or} \quad \left(r_{ik}^{(k-1)} \text{ and } r_{kj}^{(k-1)} \right)$$

Warshall's Algorithm (matrix generation)

It implies the following rules for generating $R^{(k)}$ from $R^{(k\text{-}1)}$:

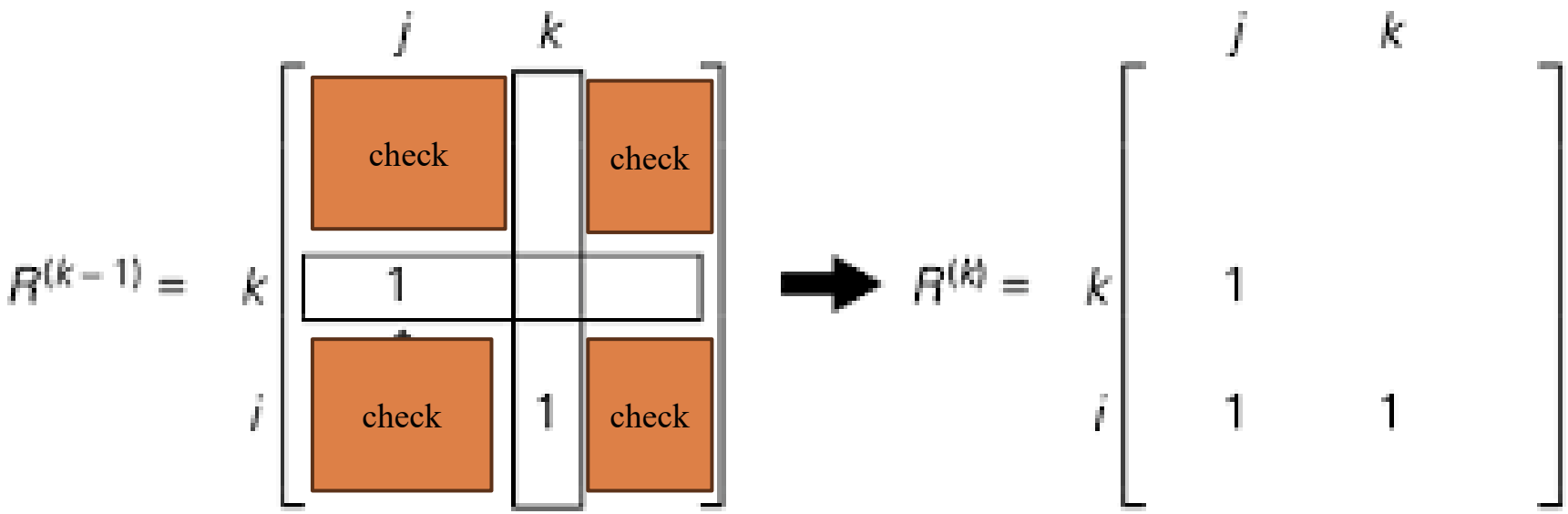
- If an element r_{ij} is 1 in $R^{(k-1)}$, it remains 1 in $R^{(k)}$.
- If an element r_{ij} is 0 in $R^{(k-1)}$, it has to be changed to 1 in $R^{(k)}$ if and only if the element in its row i and column k and the element in its column j and row k are both 1's in $R^{(k-1)}$.

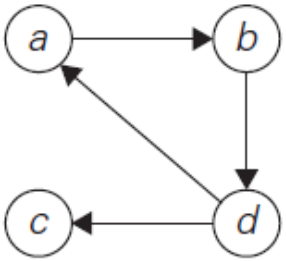


https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQxwYOg62WBPZXcPqzVz-615A19ESWpXNDvCQ&s

Warshall's Algorithm (matrix generation)

- When a vertex is concerned check the other parts of the matrix





$$R^{(0)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

1's reflect the existence of paths with no intermediate vertices ($R^{(0)}$ is just the adjacency matrix); boxed row and column are used for getting $R^{(1)}$.

$$R^{(1)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & \mathbf{1} & 1 & 0 \end{bmatrix} \end{array}$$

1's reflect the existence of paths with intermediate vertices numbered not higher than 1, i.e., just vertex *a* (note a new path from *d* to *b*); boxed row and column are used for getting $R^{(2)}$.

$$R^{(2)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & \mathbf{1} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & \mathbf{1} \end{bmatrix} \end{array}$$

1's reflect the existence of paths with intermediate vertices numbered not higher than 2, i.e., *a* and *b* (note two new paths); boxed row and column are used for getting $R^{(3)}$.

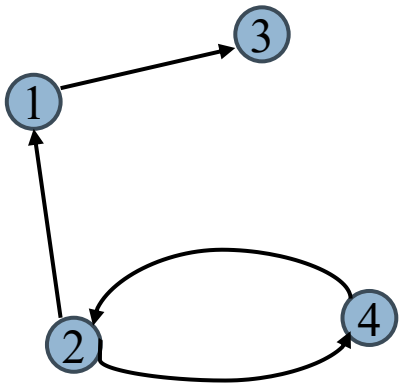
$$R^{(3)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{array}$$

1's reflect the existence of paths with intermediate vertices numbered not higher than 3, i.e., *a, b,* and *c* (no new paths); boxed row and column are used for getting $R^{(4)}$.

$$R^{(4)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} \mathbf{1} & 1 & \mathbf{1} & 1 \\ \mathbf{1} & \mathbf{1} & \mathbf{1} & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{array}$$

1's reflect the existence of paths with intermediate vertices numbered not higher than 4, i.e., *a, b, c,* and *d* (note five new paths).

Warshall's Algorithm (example)



$$R^{(0)} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \qquad R^{(1)} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & \mathbf{1} & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$R^{(2)} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ \mathbf{1} & 1 & \mathbf{1} & \mathbf{1} \end{bmatrix} \qquad R^{(3)} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \qquad R^{(4)} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & \mathbf{1} & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Transitive Closure

Warshall's Algorithm (pseudocode and analysis)

ALGORITHM *Warshall*$(A[1..n, 1..n])$

```
//Implements Warshall's algorithm for computing the transitive closure
//Input: The adjacency matrix A of a digraph with n vertices
//Output: The transitive closure of the digraph
R^(0) ← A
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            R^(k)[i, j] ← R^(k-1)[i, j] or (R^(k-1)[i, k] and R^(k-1)[k, j])
return R^(n)
```

Time efficiency: $\Theta(n^3)$

Space efficiency: Matrices can be written over their predecessors $\Theta(n^2)$

Floyd's Algorithm: All pairs shortest paths

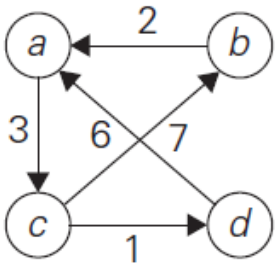
- Floyd's algorithm computes the shortest distance between every pair of vertices in a weighted directed graph
- Works with negative edge weights too, but not if there's a negative cycle.
- Given a weight matrix W for a graph with n vertices where
  - $W[i][j]$ =edge weight from i to j (use ∞ if no direct edge),
  - $W[i][i] = 0$,
- Floyd–Warshall computes $D[i][j]$ =length of the shortest path from i to j for all i, j.

Floyd's Algorithm: All pairs shortest paths

Problem: In a weighted (di)graph, find the shortest paths between every pair of vertices

Same idea: construct solution through series of matrices $D^{(0)}, \ldots, D^{(n)}$ using increasing subsets of the vertices allowed as intermediate

- $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$



(a)

$$W = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{array}$$

(b)

$$D = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ 7 & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{array}$$

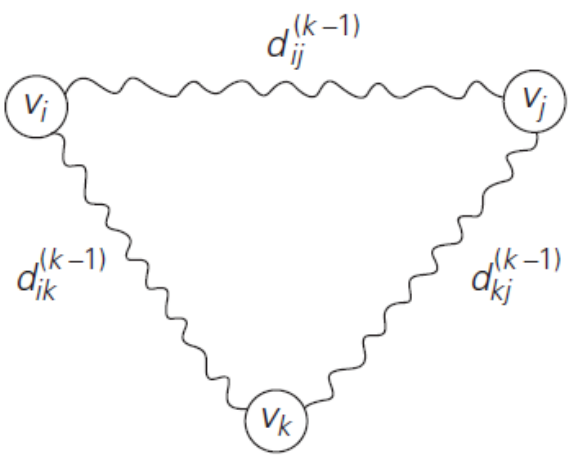
(c)

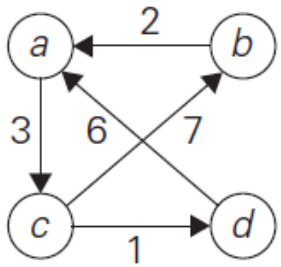
(a) Digraph matrix. (b) Its the weight matrix (c) Its distance

Floyd's Algorithm (matrix generation)

On the k-th iteration, the algorithm determines shortest paths between every pair of vertices i, j that use only vertices among 1,…,k as intermediate

$$d_{ij}^{(k)} = \min\{d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} \quad \text{for } k \geq 1, \quad d_{ij}^{(0)} = w_{ij}.$$





$$D^{(0)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{array}$$

Lengths of the shortest paths with no intermediate vertices ($D^{(0)}$ is simply the weight matrix).

$$D^{(1)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \mathbf{5} & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \mathbf{9} & 0 \end{bmatrix} \end{array}$$

Lengths of the shortest paths with intermediate vertices numbered not higher than 1, i.e., just *a* (note two new shortest paths from *b* to *c* and from *d* to *c*).

$$D^{(2)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \mathbf{9} & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{bmatrix} \end{array}$$

Lengths of the shortest paths with intermediate vertices numbered not higher than 2, i.e., a and *b* (note a new shortest path from *c* to *a*).

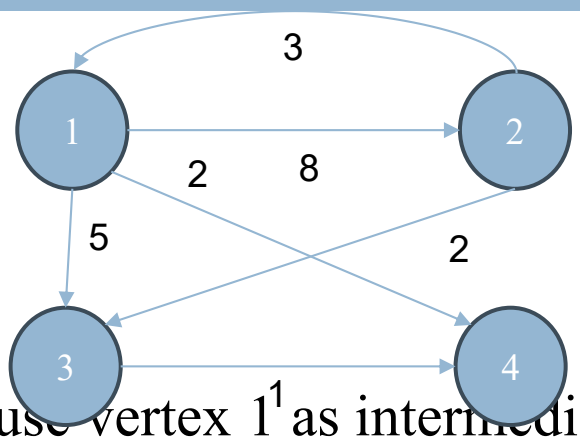
$$D^{(3)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & \mathbf{10} & 3 & \mathbf{4} \\ 2 & 0 & 5 & \mathbf{6} \\ 9 & 7 & 0 & 1 \\ 6 & \mathbf{16} & 9 & 0 \end{bmatrix} \end{array}$$

Lengths of the shortest paths with intermediate vertices numbered not higher than 3, i.e., a, *b*, and *c* (note four new shortest paths from *a* to *b*, from a to *d*, from *b* to *d*, and from *d* to *b*).

$$D^{(4)} = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 10 & 3 & 4 \\ 2 & 0 & 5 & 6 \\ \mathbf{7} & 7 & 0 & 1 \\ 6 & 16 & 9 & 0 \end{bmatrix} \end{array}$$

Lengths of the shortest paths with intermediate vertices numbered not higher than 4, i.e., a, *b*, *c*, and *d* (note a new shortest path from *c* to *a*).

Step By Step Example



- After $k = 0$(initial):
- $D^{(0)} = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix}$
- After $k = 1$(allow paths that may use vertex 1 as intermediate):
- $D^{(1)} = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{bmatrix}$
- *Examples of updates:*
  $D[2][4]$became $D[2][1] + D[1][4] = 8 + 7 = 15.$
  $D[3][2]$became $D[3][1] + D[1][2] = 5 + 3 = 8.$

Step By Step Example

- After $k = 2$:
- $D^{(2)} = \begin{bmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$
- *Key update:* $D[1][3]$becomes $D[1][2] + D[2][3] = 3 + 2 = 5$. $D[4][3]$becomes $D[4][1] + D[1][3] = 2 + 5 = 7$.
- After $k = 3$:
- $D^{(3)} = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$
- *Examples:* $D[1][4]$updated via vertex 3: $D[1][3] + D[3][4] = 5 + 1 = 6$. $D[2][4]$becomes $D[2][3] + D[3][4] = 2 + 1 = 3$ .Also $D[2][1]$improved to 7 using a path 2→3→1 possibly.

Step By Step Example

- After $k = 4$(final):
- $D^{(4)} = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$
- This final matrix gives the shortest distances between every pair.

Floyd's Algorithm (pseudocode and analysis)

ALGORITHM *Floyd*($W[1..n, 1..n]$)

```
//Implements Floyd's algorithm for the all-pairs shortest-paths problem
//Input: The weight matrix W of a graph with no negative-length cycle
//Output: The distance matrix of the shortest paths' lengths
D ← W  //is not necessary if W can be overwritten
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            D[i, j] ← min{D[i, j], D[i, k] + D[k, j]}
return D
```

Time efficiency: $\Theta(n^3)$

Space efficiency: Matrices can be written over their predecessors

Note: Shortest paths themselves can be found, too (Problem 10)

Recovering actual paths

- To reconstruct paths (not just distances), maintain a next/predecessor matrix
- next[i][j] = j if edge $i \rightarrow j$i→j exists;
- when you update D[i][j] via kk,
- set next[i][j] = next[i][k]).
- After the algorithm, reconstruct the path by following next pointers.

Reconstructing the path

- When you improve a distance using an intermediate k, i.e.,
- $D[i][j] > D[i][k] + D[k][j]$
- you **also update the path**:
- $\text{next}[i][j] = next[i][k]$
- This means:
- If the best path from i to j goes through k,
- then the *first step* from i to j is the same as the first step from i to k.

Questions